

Reference

Team: Racso va a caer. Acetilsalilic, Xook.Vector, rafaelstolopez

16 de mayo de 2026

Contents

1	Language stuff	1
1.1	Libraries and stuff	1
1.2	Compiling	1
1.3	String reading	1
1.3.1	Reading lines	1
1.3.2	String streams	1
1.3.3	Appending to string	1
1.4	Upper and lower bound	2
1.5	Primitive types	2
2	Algorithms	2
2.1	Segment tree	2
2.1.1	Recursive SegTree	2
2.2	GCD	2

2.3	Binpow	2
2.4	Prim - MST	3
2.5	Dijkstra - Minimum weight path	3
2.6	Kahn - Topo Sort	3
3	Data Structures	3
3.1	DSU	3
3.2	Ordered Set	4
3.2.1	How to use it	4
4	Maths	4
4.1	Sum of divisors	4
4.2	Hashing	4
4.3	Bitwise	4
4.3.1	Properties	4
4.3.2	Equations	4
4.4	Fermat's Little Theorem	4

1

Language stuff

1.1. Libraries and stuff

```
#include <bits/stdc++.h>
using namespace std;

// If no bits available
#include <iostream>
#include <algorithm>
#include <vector>
#include <map>
#include <set>
// iostream
#include <sstream>
// Math stuff
#include <cmath>

// fastio
#define fastio \
    ios::sync_with_stdio(0); \
    cin.tie(0); \
    cout.tie(0);
```

1.2. Compiling

```
# For version
g++ --version

# To compile
# Or 17, 20...
g++ -std=c++11 -o main <source file>

# To pipe testcases to binary
./main < text.txt
```

1.3. String reading

1.3.1. Reading lines

```
#include <iostream>
using namespace std;
int main() {
    string line;
    // Stores the whole line in "line",
    // without the newline \n
    getline(cin, line);
    // ignore the newline in the buffer
    cin.ignore();
}
```

1.3.2. String streams

```
#include <sstream>
using namespace std;
int main() {
    string a;
    // Allows to put stuff into oss,
    // built from a
    ostringstream oss(a);

    oss << 5;

    string ans1 = oss.str();
}
```

1.3.3. Appending to string

```
#include <string>
using namespace std;
int main() {
    string s;
```

```

string a;

// joins two strings (appends)
s.append(a);

// puts a character
s.push_back('a');
}

```

1.4. Upper and lower bound

- `upper_bound(a.begin(), a.end(), k)`: Returns an iterator for the first element e that is **strictly greater** than k . In other words, $e > k$.

- `lower_bound(a.begin(), a.end(), k)`: Returns an iterator for the first element e that is **greater or equal** than k . In other words, $a \geq k$.

Note: never use it over a set. Set has it's own implementation as a method: `set::lower_bound(k)`.

1.5. Primitive types

keyword	Bytes	M_{10}	m_{10}	M_2	m_2
int	4	$2 \cdot 10^9$	$-2 \cdot 10^9$	$2^{31} - 1$	-2^{31}
long long	8	$2^{63} - 1$	-2^{63}	$9 \cdot 10^{18}$	$-9 \cdot 10^{18}$

2 Algorithms

2.1. Segment tree

2.1.1. Recursive SegTree

```

#include <bits/stdc++.h>

using namespace std;

struct Node {
    int value = INT_MAX;
};

Node operate(Node a, Node b) {
    return {min(a.value, b.value)};
}

int n, q;
vector<Node> nums, tree;

void build(int index, int low, int high) {
    if(low == high) {
        tree[index].value = nums[low].value;
        return;
    }

    int mid = low + (high - low)/2;
    build(index<<1, low, mid);
    build(index<<1^1, mid+1, high);

    tree[index] = operate(tree[index<<1], tree[index<<1^1]);
}

Node query(
    int a, int b,
    int index = 1,
    int low = 0,
    int high = n-1) {
    if(high < a || b < low)
        return {INT_MAX};

    if(a <= low && high <= b)
        return tree[index];

    int mid = low + (high - low)/2;
    Node left = query(a, b, index<<1, low, mid);
    Node right = query(a, b, index<<1^1, mid+1, high);
}

```

```

    return operate(left, right);
}

void update(
    int pos,
    int val,
    int index = 1,
    int low = 0,
    int high = n-1) {
    if(low == high) {
        tree[index].value = val;
        return;
    }

    int mid = low + (high - low)/2;

    if(pos <= mid)
        update(pos, val, index<<1, low, mid);
    else
        update(pos, val, index<<1^1, mid+1, high);

    tree[index] = operate(tree[index<<1], tree[index<<1^1]);
}

void _solve() {
    int type;
    cin >> type;

    if(type == 1) {
        int k, u;
        cin >> k >> u;
        update(k-1, u);
    } else {
        int a, b;
        cin >> a >> b;

        Node ans = query(a-1, b-1);

        cout << ans.value << "\n";
    }
}

void solve() {
    cin >> n >> q;

    nums.assign(n, {});
    tree.assign(4*n + 1, {INT_MAX});

    for(Node& i : nums) cin >> i.value;

    build(1, 0, n-1);

    while(q--)
        _solve();
}

int main() {
    ios.base::sync_with_stdio(false);
    cin.tie(NULL);
    cout.tie(NULL);

    solve();

    return 0;
}

```

2.2. GCD

```

long long gcd(long long a, long long b)
{
    if(b == 0) return a;

    return gcd(b, a % b);
}

```

2.3. Binpow

```

const int MOD = 1e9 + 7;

long long binpow(long long x, long long n)
{
    long long ans = 1;
}

```

```

while(n > 0)
{
    if(n & 1)
        ans = (ans * x) % MOD;

    x = (x * x) % MOD;
    n >>= 2;
}

return ans % MOD;
}

```

2.4. Prim - MST

```

#include <vector>
#include <queue>
#include <set>
using namespace std;
int main() {
    int n, m;
    vector<vector<int>>> g(n + 1); // Adjacency matrix
    vector<int> values(n + 1); // The indegree of each node

    // This stores nodes already eliminated from the graph
    set<int> elimin;
    // Stores the order in which each node is eliminated
    queue<int> order;
    // The next node to visit - normal BFS
    queue<int> next;

    // Need to record the indegree of each node
    for (int i=1; i<=n; i++) {
        for (int v : g[i])
            values[v]++;
    }

    // If indegree[i] == 0, then we can start there
    for (int i=1; i<=n; i++)
        if (!values[i])
            next.push(i);

    // Start the BFS
    while (!next.empty()) {
        int curr = next.front();
        next.pop();
        // If already got rid of it, then no need to revisit
        if (elimin.find(curr) != elimin.end())
            continue;
        elimin.insert(curr);

        // Reduce the indegree of each neighbour, as we
        // eliminated the current node
        for (int neigh : g[curr]) {
            values[neigh]--;

            // If indegree[neigh] == 0, then we can continue
            // to it, cause we have none pending
            if (!values[neigh])
                next.push(neigh);
        }
        // We need to store the currently eliminated node
        // into the final order
        order.push(curr);
    }

    if (order.size() < n);
    // In this case, we haven't visited all nodes. This
    // can mean:
    // 1. There's more than one connected component or
    // 2. There's a cycle
    else;
    // Here we have a valid order for the nodes of our graph
    // into order. We can do as we wish.
}

```

2.5. Dijkstra - Minimum weight path

```

#include <vector>
#include <queue>
#include <limits>
using namespace std;
long long dijkstra(
    vector<vector<pair<int, int>>> const& adj,
    int start, int goal

```

```

)
{
    int n = adj.size();
    vector<long long> distance(n+1, LLONG_MAX);

    priority_queue<
        pair<long long, long long>,
        vector<pair<long long, long long>>,
        greater<pair<long long, long long>>>
    > next;

    // {cost, node}
    next.push({0, start});

    while (!next.empty())
    {
        auto [weight, node] = next.top();
        next.pop();

        if (weight >= distance[node])
            continue;

        distance[node] = weight;

        for (auto [nweight, nnode] : adj[node])
        {
            if (nweight + distance[node] >= distance[nnode])
                continue;
            next.push({nweight + distance[node], nnode});
        }
    }
    return distance[goal];
}

```

2.6. Kahn - Topo Sort

```

#include <bits/stdc++.h>
using namespace std;
int V; // Number of vertices
int E; // Number of edges
vector<int> indegree(V+1, 0);

vector<int> kahn(vector<vector<int>> const& graph)
{
    vector<int> order, deg = indegree; // copy
    queue<int> q;
    for(int i = 1; i <= V; i++)
        if(deg[i] == 0)
            q.push(i);
    while(!q.empty()) {
        int u = q.front(); q.pop();
        order.push_back(u);
        for(int v : graph[u]) {
            deg[v]--;
            if(deg[v] == 0)
                q.push(v);
        }
    }
    return order;
};

```

3 Data Structures

3.1. DSU

```

#include <bits/stdc++.h>
using namespace std;
vector<int> parent;
vector<int> _size;

void make_set(int v) {
    parent[v] = v;
    _size[v] = 1;
}

int find_set(int v) {
    if (v == parent[v])
        return v;
    return parent[v] = find_set(parent[v]);
}

// Union by size/rank - small to large

```

```
void union_sets(int a, int b) {
    a = find_set(a);
    b = find_set(b);
    if (a != b) {
        if (_size[a] < _size[b])
            swap(a, b);
        parent[b] = a;
        _size[a] += _size[b];
    }
}
```

```
void sum_all_divisors(int n, vector<long long>& sigma)
{
    sigma.assign(n+1, 1);

    for(int i = 2; i <= n; i++)
        for(int j = i; j <= n; j += i)
            sigma[j] += i;
}
```

3.2. Ordered Set

```
// Common file
#include <ext/pb_ds/assoc.container.hpp>
// Including tree_order_statistics_node_update
#include <ext/pb_ds/tree_policy.hpp>

using namespace std;
using namespace __gnu_pbds;

typedef tree<
    int,
    nulltype,
    less<int>,
    rb_tree_tag,
    tree_order_statistics_node_update>
    ordered_set;
```

3.2.1. How to use it

- `o_set::order_of_key(k)`: Gets the number of elements that are **strictly smaller** than k . $e < k$. If k is greater than every element in the set, then it returns `set.size()`.
- `o_set::find_by_order(n)`: Returns an iterator to the n -th element counting from zero. That is, gets the element that has a `order_of_key` of n . If no such element exists, i.e. the requested element is too big, then `set.end()` is returned.

4
Maths

4.1. Sum of divisors

4.2. Hashing

$$H(s) = (s_i p^{i-1} + s_{i+1} p^i + \dots) \% M = \sum_{i=1}^{|s|} s_i p^{i-1} \% M$$

$$M_1 = 127657753, \quad p_1 = 137$$

$$M_2 = 987654319, \quad p_2 = 277$$

4.3. Bitwise

4.3.1. Properties

$$\begin{aligned} a|b &= a \oplus (a \& b) \\ a \oplus (a \& b) &= (a|b) \oplus b \\ b \oplus (a \& b) &= (a|b) \oplus a \\ (a \& b) \oplus (a|b) &= a \oplus b \end{aligned}$$

4.3.2. Equations

$$\begin{aligned} a + b &= (a|b) + (a \& b) \\ a + b &= a \oplus b + 2(a \& b) \\ a - b &= (a \oplus (a \& b)) - ((a|b) \oplus a) \\ a - b &= ((a|b) \oplus b) - ((a|b) \oplus a) \\ a - b &= (a \oplus (a \& b)) - (b \oplus (a \& b)) \\ a - b &= ((a|b) \oplus b) - (b \oplus (a \& b)) \end{aligned}$$

4.4. Fermat's Little Theorem

$$a^{p-1} \equiv 1 \pmod{p} \implies \gcd(a, p) = 1$$